

Deployment Report

Docker

Per Selasa, 22 Juli 2025, kelompok kami telah berhasil melakukan deployment ketiga komponen aplikasi (backend, frontend, dan verifikator) dalam 1 buah deployment yang sama menggunakan `docker-compose.yml`.

Adapun struktur proyeknya dibuat sebagai berikut:

```
.
├── docker-compose.yml
├── backend-secure-onboarding-system
│   ├── Dockerfile
├── frontend-secure-onboarding-system
│   ├── Dockerfile
├── ops-secure-onboarding-system
│   ├── Dockerfile
└── verifikator-secure-onboarding-system
    ├── Dockerfile
```

Berikut adalah implementasi masing-masingnya :

- `docker-compose.yml`

```
services:
  # Database untuk Backend
  db_backend:
    image: postgres:17
    container_name: db_main
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: customer_registration
    volumes:
      # Memasang skrip inisialisasi database dari folder backend
      - db_backend_data:/var/lib/postgresql/data
      - ./backend-secure-onboarding-
system/sql/database_setup_DOCKER.sql:/docker-entrypoint-initdb.d/init.sql
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d
customer_registration"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
```

```

    - secure-onboarding-network

# Database untuk Verifikator
db_verifikator:
  image: postgres:17
  container_name: db_validator
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: password
    POSTGRES_DB: dukcapil_ktp
  volumes:
    # Memasang skrip inisialisasi database dari folder verifikator
    - db_verifikator_data:/var/lib/postgresql/data
    - ./verifikator-secure-onboarding-
system/sql/database_DOCKER.sql:/docker-entrypoint-initdb.d/init.sql
  ports:
    - "5434:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres -d dukcapil_ktp"]
    interval: 5s
    timeout: 5s
    retries: 5
  networks:
    - secure-onboarding-network

# Layanan Backend
backend:
  build:
    context: ./backend-secure-onboarding-system # Konteks build adalah
direktori layanan backend
    dockerfile: Dockerfile # Dockerfile bernama 'Dockerfile' di dalam
direktori konteks
  container_name: backend
  ports:
    - "8083:8080"
  environment:
    SPRING_DATASOURCE_URL:
jdbc:postgresql://db_main:5432/customer_registration
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: password
    SERVER_PORT: 8080
    JWT_SECRET:
superSecretKeyForHS512AlgorithmThatIsAtLeastSixtyFourCharactersLongForStron
gSecurity12345!@#$$%^&*()
    JWT_EXPIRATION: 86400000
  depends_on:
    db_backend:
      condition: service_healthy
  networks:
    - secure-onboarding-network

# Layanan Verifikator
verifikator:
  build:

```

```

    context: ./verificator-secure-onboarding-system # Konteks build
    adalah direktori layanan verifikator
    dockerfile: Dockerfile # Dockerfile bernama 'Dockerfile' di dalam
    direktori konteks
    container_name: external_validator
    ports:
      - "8082:8081"
    environment:
      SPRING_DATASOURCE_URL:
jdbc:postgresql://db_validator:5432/dukcapil_ktp
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: password
      SERVER_PORT: 8081
    depends_on:
      db_verifikator:
        condition: service_healthy
    networks:
      - secure-onboarding-network

# Layanan Frontend
frontend:
  build:
    context: ./frontend-secure-onboarding-system # Konteks build adalah
    direktori layanan frontend
    dockerfile: Dockerfile # Dockerfile bernama 'Dockerfile' di dalam
    direktori konteks
    container_name: frontend
    ports:
      - "3000:80"
    environment:
      VITE_API_BASE_URL_BACKEND: http://localhost:8083
      VITE_API_BASE_URL_VERIFICATOR: http://localhost:8082
    depends_on:
      - backend
      - verifikator
    networks:
      - secure-onboarding-network

volumes:
  db_backend_data:
  db_verifikator_data:

networks:
  secure-onboarding-network:
    driver: bridge

```

- backend-secure-onboarding-system/Dockerfile

```

# Stage 1: Build aplikasi menggunakan Maven
# Gunakan image Maven + Java 21
FROM maven:3.9.6-eclipse-temurin-21 AS build

```

```
# Set working directory di dalam container build stage
WORKDIR /app

# Copy pom.xml dan mvnw (Maven Wrapper) terlebih dahulu untuk memanfaatkan
Docker layer caching.
# Karena Dockerfile ini berada di dalam folder backend-secure-onboarding-
system,
# jalur COPY sekarang relatif terhadap folder tersebut (yaitu, dari root
folder backend).
COPY pom.xml ./
COPY mvnw ./
COPY .mvn ./mvn

# Beri izin eksekusi pada mvnw
RUN chmod +x mvnw

# Copy seluruh source code backend
# Ini akan menyalin semua file dari folder src ke /app/src di dalam
container.
COPY src ./src

# Salin file .env ke src/main/resources agar masuk ke classpath JAR
# Ini akan memastikan bahwa file .env dapat ditemukan oleh aplikasi saat
runtime
# karena akan dikemas di dalam JAR pada lokasi yang diharapkan oleh
classpath.
COPY .env ./src/main/resources/

# Build aplikasi Spring Boot
# Perintah ini akan mengkompilasi kode dan membuat file JAR yang dapat
dieksekusi.
# -DskipTests digunakan untuk melewati pengujian selama proses build
Docker.
RUN ./mvnw clean package -DskipTests

# Stage 2: Jalankan jar dengan OpenJDK
# Gunakan image OpenJDK yang ringan untuk menjalankan aplikasi.
FROM eclipse-temurin:21-jdk-alpine
    # Menggunakan alpine untuk ukuran image yang lebih kecil

# Set working directory di dalam container runtime stage
WORKDIR /app

# Copy hasil build (file JAR) dari stage 'build' ke direktori /app di stage
ini.
# File JAR yang dihasilkan akan memiliki nama seperti 'secure-onboarding-
system-0.0.1-SNAPSHOT.jar'
# atau nama lain sesuai konfigurasi di pom.xml Anda.
# Kita menyalinnya sebagai 'app.jar' untuk konsistensi.
COPY --from=build /app/target/*.jar app.jar

# --- Opsional: Tambahkan baris ini untuk debugging (misalnya, untuk
netcat) ---
# eclipse-temurin:21-jdk-alpine adalah berbasis Alpine, jadi gunakan apk
```

```
# RUN apk update && apk add netcat-traditional && rm -rf /var/cache/apk/*
# Baris di atas akan menginstal netcat untuk debugging jika diperlukan.
# 'rm -rf /var/cache/apk/*' membersihkan cache apk untuk menjaga ukuran
image tetap kecil.
# -----

# Tentukan perintah yang akan dijalankan saat container dimulai.
# Ini akan menjalankan aplikasi Spring Boot Anda.
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

- frontend-secure-onboarding-system/Dockerfile

```
# Stage Pertama: Build Aplikasi Frontend
# Gunakan base image Node.js yang sesuai (misalnya Node.js 20)
FROM node:20-alpine AS build

# Set working directory
WORKDIR /app

# Copy package.json dan package-lock.json (jika ada) terlebih dahulu.
# Ini memanfaatkan Docker layer caching: jika file-file ini tidak berubah,
# npm install tidak akan dijalankan ulang di build berikutnya, mempercepat
proses.
# Karena Dockerfile ini berada di dalam folder frontend-secure-onboarding-
system,
# jalur COPY sekarang relatif terhadap folder tersebut.
COPY package.json ./
COPY package-lock.json ./

# Install dependensi
RUN npm install

# Copy seluruh source code frontend
# Pastikan semua file proyek (termasuk public, src, dll.) disalin setelah
dependensi diinstal.
# Karena Dockerfile ini berada di dalam folder frontend-secure-onboarding-
system,
# jalur COPY sekarang relatif terhadap folder tersebut.
COPY . .

# Build aplikasi Vite/React
# Perintah ini akan menghasilkan file-file statis yang siap disajikan.
RUN npm run build

# ---

# Stage Kedua: Serving Aplikasi dengan Nginx
# Gunakan base image Nginx yang ringan
FROM nginx:alpine

# Copy hasil build dari stage 'build' ke direktori Nginx default
# '/usr/share/nginx/html' adalah lokasi default Nginx untuk file statis.
```

```
COPY --from=build /app/dist /usr/share/nginx/html

# Copy konfigurasi Nginx kustom.
# Pastikan Anda telah membuat file nginx.conf di direktori frontend-secure-
onboarding-system.
COPY nginx.conf /etc/nginx/conf.d/default.conf

# Expose port 80 untuk Nginx
# Ini memberi tahu Docker bahwa container akan mendengarkan di port ini.
EXPOSE 80

# Command untuk memulai Nginx di foreground.
# "daemon off;" memastikan Nginx tetap berjalan di foreground agar
container tidak langsung mati.
CMD ["nginx", "-g", "daemon off;"]
```

- [verificator-secure-onboarding-system/Dockerfile](#)

```
# Stage 1: Build aplikasi menggunakan Maven
# Gunakan image Maven + Java 21
FROM maven:3.9.6-eclipse-temurin-21 AS build

# Set working directory di dalam container build stage
WORKDIR /app

# Copy pom.xml dan mvnw (Maven Wrapper) terlebih dahulu untuk memanfaatkan
Docker layer caching.
# Karena Dockerfile ini berada di dalam folder verificator-secure-
onboarding-system,
# jalur COPY sekarang relatif terhadap folder tersebut.
COPY pom.xml ./
COPY mvnw ./
COPY .mvn ../.mvn

# Beri izin eksekusi pada mvnw
RUN chmod +x mvnw

# Copy seluruh source code verifikator
# Ini akan menyalin semua file dari folder src ke /app/src di dalam
container.
COPY src ./src

# Build aplikasi Spring Boot
# Perintah ini akan mengkompilasi kode dan membuat file JAR yang dapat
dieksekusi.
# -DskipTests digunakan untuk melewati pengujian selama proses build
Docker.
RUN ./mvnw clean package -DskipTests

# Stage 2: Jalankan jar dengan OpenJDK
# Gunakan image OpenJDK yang ringan untuk menjalankan aplikasi.
FROM eclipse-temurin:21-jdk-alpine
```

```

# Menggunakan alpine untuk ukuran image yang lebih kecil

# Set working directory di dalam container runtime stage
WORKDIR /app

# Copy hasil build (file JAR) dari stage 'build' ke direktori /app di stage ini.
# File JAR yang dihasilkan akan memiliki nama seperti 'verificator-secure-onboarding-system-0.0.1-SNAPSHOT.jar'
# atau nama lain sesuai konfigurasi di pom.xml Anda.
# Kita menyalinnya sebagai 'app.jar' untuk konsistensi.
COPY --from=build /app/target/*.jar app.jar

# --- Opsional: Tambahkan baris ini untuk debugging (misalnya, untuk netcat) ---
# eclipse-temurin:21-jdk-alpine adalah berbasis Alpine, jadi gunakan apk
# RUN apk update && apk add netcat-traditional && rm -rf /var/cache/apk/*
# Baris di atas akan instal netcat untuk debugging jika diperlukan.
# 'rm -rf /var/cache/apk/*' membersihkan cache apk untuk menjaga ukuran image tetap kecil.
# -----

# Tentukan perintah yang akan dijalankan saat container dimulai.
# Ini akan menjalankan aplikasi Spring Boot Anda.
ENTRYPOINT ["java", "-jar", "/app/app.jar"]

```

- [ops-secure-onboarding-system/Dockerfile](#)

```

FROM maven:3.8.7-openjdk-17 AS builder

WORKDIR /app

COPY pom.xml .
RUN mvn dependency:go-offline

COPY src ./src

RUN mvn clean install -DskipTests

FROM openjdk:17-jdk-slim

WORKDIR /app

ARG JAR_FILE=target/ops-secure-onboarding-system-0.0.1-SNAPSHOT.jar
COPY --from=builder /app/${JAR_FILE} app.jar

EXPOSE 8080

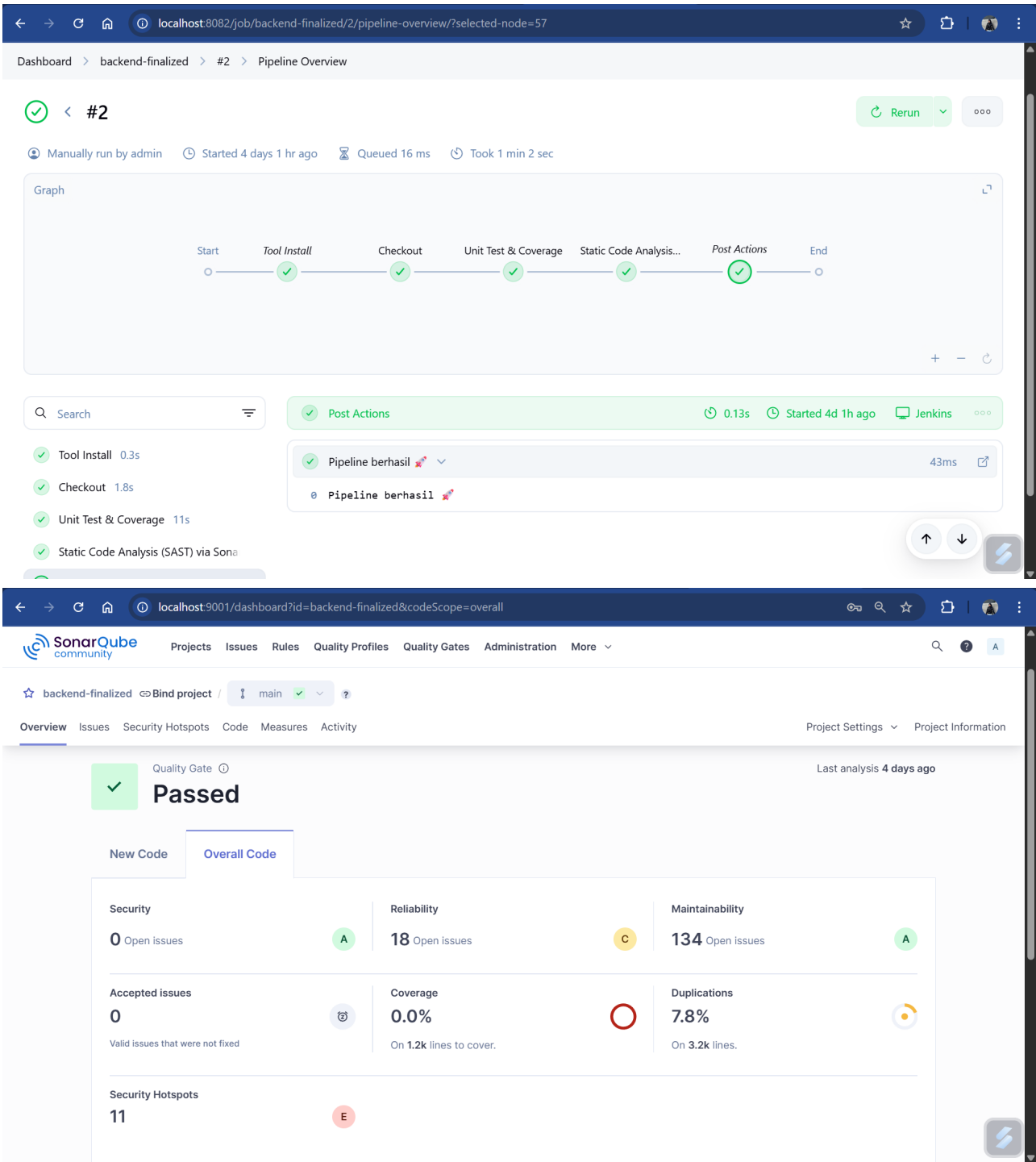
# Command to run the application
ENTRYPOINT ["java", "-jar", "app.jar"]

```

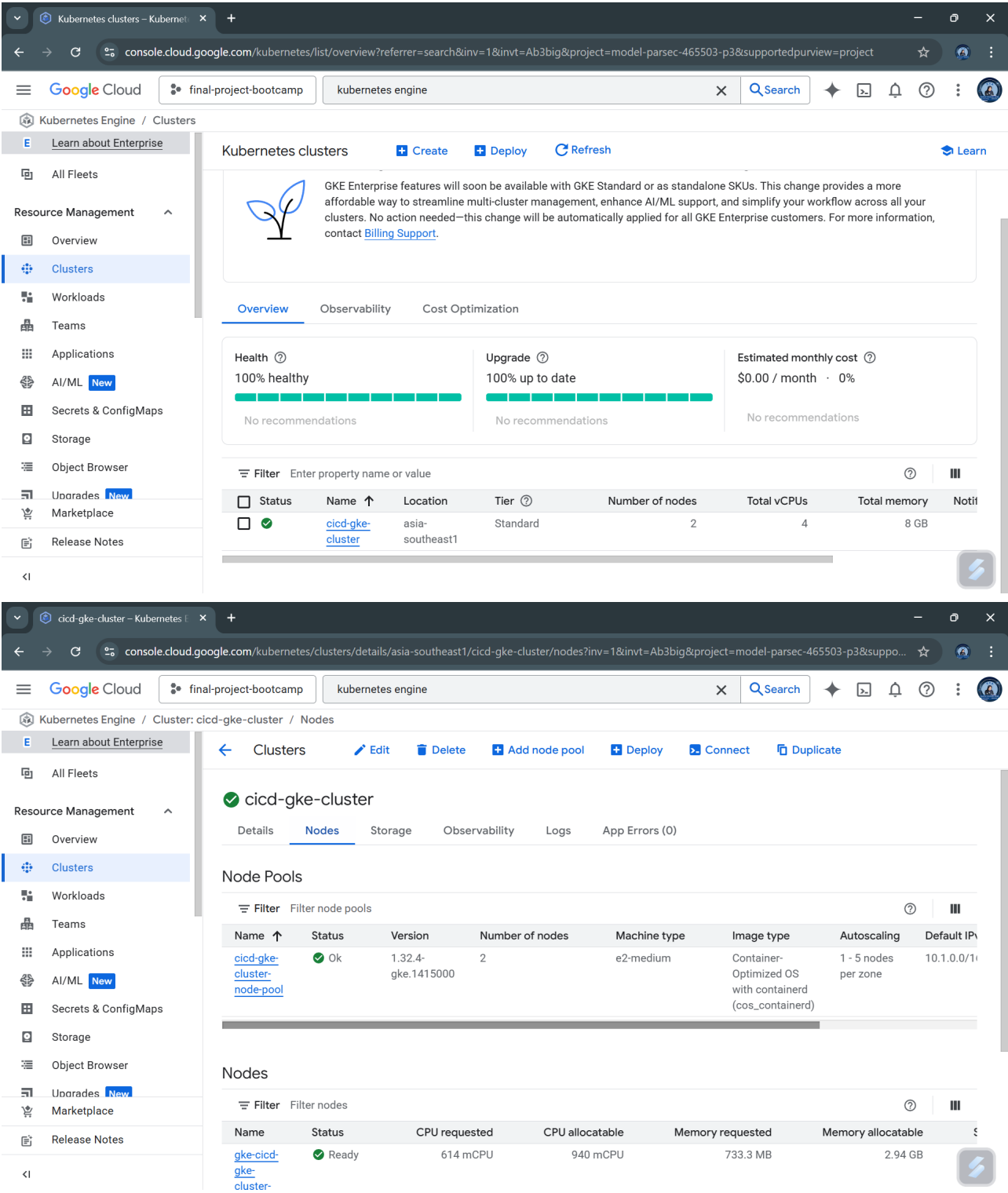
CI/CD Pipeline using Jenkins

Pada saat pengumpulan Task Week 2 (Selasa, 22 Juli 2025), kelompok kami telah berhasil

- 1. Melakukan SAST menggunakan SonarQube Berikut adalah contoh testing backend code.



- 2. Melakukan deployment Google Kubernetes Engine (GKE) dengan mengaplikasikan Terraform code. Pada saat ini kelompok kami masih mencoba melakukan deployment frontend terlebih dahulu. Deployment GKE menggunakan Terraform sudah berhasil dilakukan.



Namun, masih terdapat permasalahan pada IAM policy. Kelompok kami akan meninjau lebih lanjut

penyebab permasalahan ini.

jenkins.andrc1613.my.id/job/terraform-deployment/37/pipeline-overview/

Dashboard > terraform-deployment > #37 > Pipeline Overview

Search

Tool Install 0.15s

Checkout Frontend 3.6s

Docker Build and Push Frontend 3.3s

Checkout Terraform Configuration

Terraform Apply GKE Deployment 1m 1s

Post Actions 0.1s

terraform apply -auto-approve tfplan 12m

126 |

127 | Error:

Request `Create IAM Members roles/iam.serviceAccountUser serviceAccount:gke-node-sa@model-parsec-465503-p3.iam.gserviceaccount.com for project "model-parsec-465503-p3"` returned error: Batch request and retried single request "Create IAM Members roles/iam.serviceAccountUser serviceAccount:gke-node-sa@model-parsec-465503-p3.iam.gserviceaccount.com for project \"model-parsec-465503-p3\"\" both failed. Final error: Error applying IAM policy for project "model-parsec-465503-p3": Error setting IAM policy for project "model-parsec-465503-p3": googleapi: Error 403: Policy update access denied., forbidden

128 |

129 | with google_project_iam_member.gke_node_sa_roles["roles/iam.serviceAccountUser"],

130 | on main.tf line 89, in resource "google_project_iam_member" "gke_node_sa_roles":

131 | 89: resource "google_project_iam_member" "gke_node_sa_roles" {

132 |

133 |

134 |

135 | Error:

Request `Create IAM Members roles/compute.viewer serviceAccount:gke-node-sa@model-parsec-465503-p3.iam.gserviceaccount.com for project "model-parsec-465503-p3"` returned error: Batch request and retried single request "Create IAM Members roles/compute.viewer serviceAccount:gke-node-sa@model-parsec-465503-p3.iam.gserviceaccount.com for project \"model-parsec-465503-p3\"\" both failed. Final error: Error applying IAM policy for project "model-parsec-465503-p3": Error setting IAM policy for project "model-parsec-465503-p3": googleapi: Error 403: Policy update access denied., forbidden